VELLORE INSTITUTE OF TECHNOLOGY, BHOPAL
Problem Solving and Programming · VITyarthi – Build Your Own Project

PROJECT REPORT

Smart Study & Exam Priority Tracker

A Python console application that ranks subjects by weighted deficit and exam proximity to produce a daily study plan.

SUBMITTED BY	Ananya Singh
REGISTRATION NO	26BEC10011
PROGRAM	B.Tech Electronics and Communication Engineering
YEAR / SEMESTER	1st Year / Fall Semester 2026–2027
SUBMITTED TO	Gunjan Ansari
SUBMITTED AS A PART OF	VITyarthi – Build Your Own Project
ACADEMIC YEAR	2026–2027

Contents

1. Introduction . . . 3
2. Problem Statement . . . 3
3. Objectives . . . 3
4. Functional Requirements . . . 4
5. Non-Functional Requirements . . . 4
6. System Architecture . . . 5
7. Design Diagrams . . . 6
8. Design Decisions & Rationale . . . 8
9. Implementation Details . . . 8
10. Screenshots / Results . . . 9
11. Testing Approach . . . 10
12. Challenges Faced . . . 10
13. Learnings & Key Takeaways . . . 10
14. Future Enhancements . . . 11
15. References . . . 11

1. Introduction

The **Smart Study & Exam Priority Tracker** is a Python-based console application designed to help students optimize their daily revision schedules based on algorithmic evaluation. Rather than relying on static or arbitrary study planning, the system gathers key metrics per subject—including previous test scores, percentage completion of practice exercises, revision history, topic coverage, and upcoming exam dates.

By calculating weighted deficit scores combined with proximity-based exam urgency multipliers, the system automatically determines priority levels (`HIGH`, `MEDIUM`, `LOW`), assigns daily study durations, and pinpoints specific focus areas. The software features modular functions, flexible data structures, and dynamic priority sorting to ensure actionable academic planning.

2. Problem Statement

Students frequently manage multiple academic subjects concurrently, each demanding different levels of attention based on approaching exam deadlines and concept mastery. Traditional study routines lack algorithmic objectivity, leading students to spend disproportionate time on strong subjects while neglecting vulnerable topics or looming deadlines.

There is a clear need for an automated decision-support utility that can intake subject-level metrics, analyze performance gaps, apply urgency scaling based on calendar proximity, and output a prioritized daily action plan.

3. Objectives

1. **Automate Priority Calculation:** Compute daily study priorities using an algorithmic formula incorporating weighted deficit metrics and exam proximity multipliers.
2. **Dynamic Focus Pinpointing:** Identify specific weak topics or critical metrics driving subject deficits to recommend focused study sessions.
3. **Structured Data Handling:** Utilize Python dictionaries and lists to store subject metrics, user inputs, and calculated outputs efficiently.
4. **Error Resilient Parsing:** Integrate date parsing logic (`datetime.strptime`) with fallback error handling for safe date math calculations.

5. **Practical Python Application:** Apply core programming constructs including loops, conditional statements, mathematical modeling, and function decomposition.

4. Functional Requirements

1. **Subject Data Input:** Collect user input for number of subjects, marks percentage, practice percentage, overall revision percentage, recent revision status, topic completion percentage, exam date (`YYYY-MM-DD`), sub-topics list, and weak topic focus.
2. **Deficit Metrics Calculation:** Compute individual weighted deficit scores across five core academic dimensions:
   - Marks Deficit (15%)
   - Practice Deficit (25%)
   - Revision Deficit (25%)
   - Last Revision Deficit (20%)
   - Topic Completion Deficit (15%)
3. **Urgency Multiplier Calculation:** Calculate proximity-based urgency factor (20.0 / days_left) bounded dynamically by exam dates.
4. **Priority & Time Allocation:** Classify total score into `HIGH` (>= 55), `MEDIUM` (>= 33), or `LOW` priority, assigning recommended daily durations (2 hours, 1 hour, or 30 minutes).
5. **Reasoning Engine:** Identify and display the primary driver of high score or acknowledge strong performance metrics.
6. **Sorted Priority Output:** Sort and display today's study plan ordered by priority status and overall composite score.

5. Non-Functional Requirements

1. **Usability:** Terminal-based console prompts are structured sequentially with explicit format guidance.
2. **Reliability:** Exception handling wraps date-string parsing to ensure invalid date inputs default safely without crashing execution.
3. **Performance:** Priority metrics calculation and sorting execute instantly (O(N log N) complexity for N subjects).

4. **Maintainability:** Pure function separation between input collection (`get_user_input`), calculation logic (`calculate_priority`), and rendering (`display_today_priority`).
5. **Portability:** Built using standard Python standard libraries (`datetime`) without external third-party library dependencies.

6. System Architecture

The architecture follows a pipeline flow: data collection -> mathematical evaluation -> priority classification -> output rendering.

Component	Description
User Input Module	Prompts user for subject parameters and parses sub-topics and weak areas.
Deficit Calculation Engine	Applies weighted multipliers to compute performance deficits across 5 key areas.
Urgency Evaluator	Calculates day difference using `datetime.date` objects and scales deficit by proximity factor.
Priority Classifier	Thresholds score into HIGH, MEDIUM, or LOW and assigns target study hours.
Output Generator	Sorts subjects by priority level and score, presenting an actionable study plan.

7. Design Diagrams

7.1 Use Case Diagram

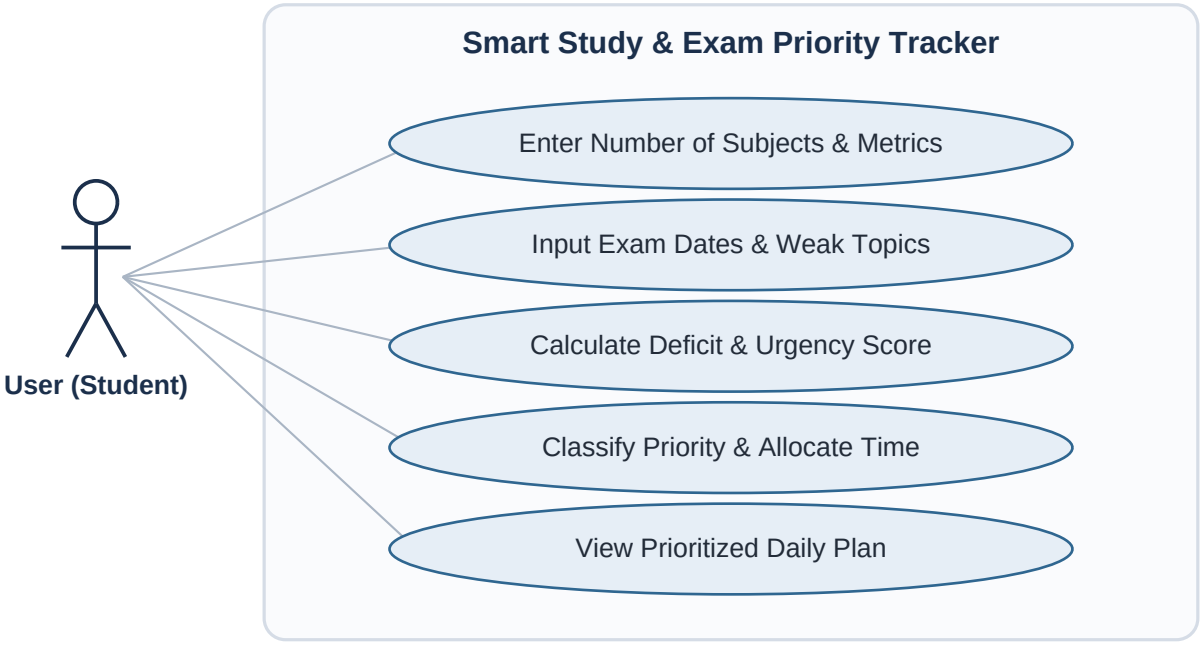

Figure 1. User interactions with the tracker.

7.2 Workflow Diagram

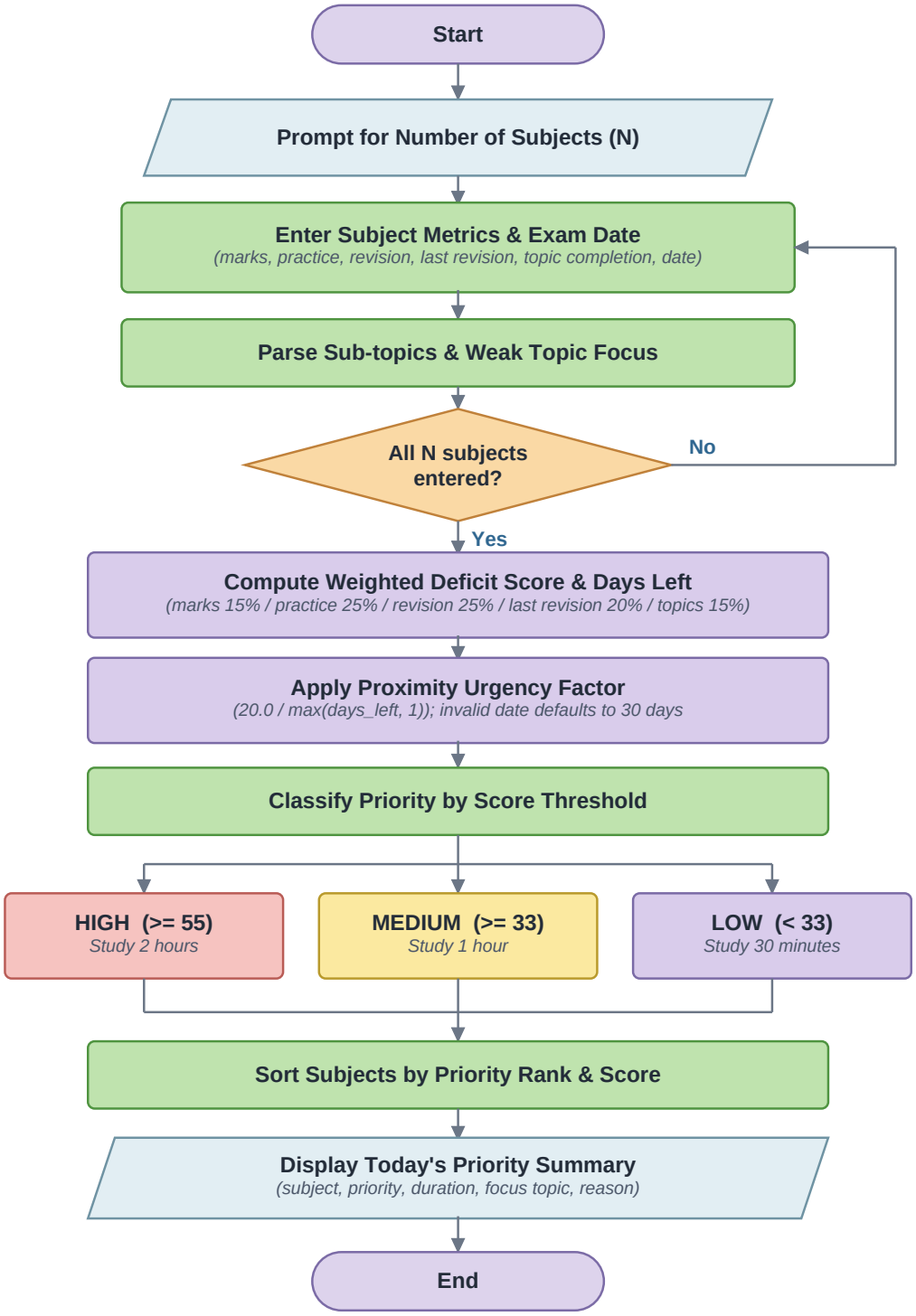

Figure 2. End-to-end workflow, from input collection to the prioritized daily plan.

8. Design Decisions & Rationale

1. **Weighted Deficit Evaluation:** Weighted coefficients (Practice 25%, Revision 25%, Last Revision 20%, Marks 15%, Topic Completion 15%) prioritize practice and active recall while balancing past performance.
2. **Proximity Urgency Scaling:** Dividing by remaining exam days (20.0 / days_left) exponentializes priority as deadlines approach.
3. **Data Structure Selection:** Dictionaries represent individual subject instances with explicit field keys, while standard lists store subject records for batch processing.
4. **Graceful Exception Fallback:** Wrapping date parsing in `try-except ValueError` blocks prevents runtime failures on bad inputs by applying a safe default (30 days left).

9. Implementation Details

1. **Language & Environment:** Developed in Python 3 using VS Code terminal environment.
2. **Modules & Dependencies:** Uses built-in `datetime.datetime` and `datetime.date` modules.
3. **Key Concepts Applied:**
   - Function Decomposition (`get_user_input`, `calculate_priority`, `display_today_priority`)
   - String parsing and list comprehensions (`split(",")`, strip sanitation)
   - Mathematical logic and bounding (`max(days_left, 1)`, weighted formulas)
   - Multi-key lambda sorting (`key=lambda x: (priority_order[x["priority"]], x["score"])`)

10. Screenshots / Results

10.1 CLI Execution Sample

```
How many subjects do you want to add? 2
Enter the subject name: Mathematics
Enter previous marks or quiz score (%): 65
Enter practice completion (%): 40
Enter revision completion (%): 50
Enter last revision completion (%): 30
Enter overall topic completion (%): 60
Enter exam date (YYYY-MM-DD): 2026-10-05
Enter 2 or 3 topics for this subject (separated by ,
Topics: Calculus, Linear Algebra, Probability
Which topic needs the most attention in Mathematics? Calculus

Enter the subject name: Computer Networks
Enter previous marks or quiz score (%): 85
Enter practice completion (%): 90
Enter revision completion (%): 85
Enter last revision completion (%): 80
Enter overall topic completion (%): 90
Enter exam date (YYYY-MM-DD): 2026-11-15
Enter 2 or 3 topics for this subject (separated by ,
Topics: Routing, Subnetting, TCP/IP
Which topic needs the most attention in Computer Networks? Subnetting

PRIORITY for today
Subject: Mathematics
Priority: HIGH
Required study: 2 hours
Focus topic: Calculus
Reason: Practice_incomplete + upcoming exam (6 days left)

Subject: Computer Networks
Priority: LOW
Required study: 30 minutes
Focus topic: Subnetting
Reason: Good marks + high revision
```

11. Testing Approach

Test Case	Scenario / Input	Expected Result	Status
TC01	Standard Subject Metrics Input	Successfully stores values in subject dictionary	Pass
TC02	Exam Date < 7 Days	Score scales up; reason reflects upcoming exam	Pass
TC03	Invalid Date Format (e.g., `12-10-2026`)	Caught by `ValueError`, defaults `days_left = 30`	Pass
TC04	Empty Weak Topic String	Fallback selects first topic or "General Revision"	Pass
TC05	High Score Metric Calculation	Correctly categorizes as HIGH priority with 2 hours study	Pass
TC06	Priority Sorting Output	Displays HIGH priority subjects before MEDIUM and LOW	Pass

12. Challenges Faced

1. **Date Math Operations:** Safely computing day differentials between dynamic string user inputs and `date.today()` required handling standard formatting and past dates.
2. **Dynamic Key Extraction:** Programmatically determining the highest metric contributor dynamically required mapping dictionary definitions to `max()` evaluation functions.
3. **Multi-tier Sorting Logic:** Custom sorting required a composite tuple key `(priority_level_rank, raw_score)` to guarantee ordering consistency.

13. Learnings & Key Takeaways

1. Applied computational problem-solving principles to convert multi-factor criteria into priority scores.
2. Practical usage of Python's `datetime` library for time-based evaluation routines.
3. Structured code organization into modular functions for calculation, data input, and presentation.

4. Defensive programming habits using validation checks and default value fallbacks.

14. Future Enhancements

1. **Data Persistence:** Save student progress and subject logs into local JSON or SQLite storage across sessions.
2. **GUI / Web Interface:** Build an interactive frontend using Streamlit or Tkinter.
3. **Calendar Integration:** Export study schedules directly to `.ics` files for sync with Google Calendar or Outlook.
4. **Historical Analytics:** Plot learning curve progress and practice score improvements over time using graphical visualizers.

15. References

1. Python Software Foundation. *Python Documentation – datetime module.*
2. Visual Studio Code Documentation. *Python Development in VS Code.*
3. VITyarthi. *Build Your Own Project Course Guidelines & Evaluation Rubric.*